AI Assisted Coding Lab 20.3

Veekshitha Adharasani

2303A52175

Batch – 41

Question 1: Password Strength & Validation Check

Prompt:

Generate a python code for user registration program. Check whether password length is validate and weak passwords are allowed.

Code:

```python
# Generate a python code for user registration program. Check whether password length is validate and weak passwords are allowed.
def is_valid_password(password):
    """
    Check if the provided password is valid based on length and strength criteria.
    A valid password must be at least 8 characters long. Weak passwords are allowed,
    but a warning will be issued if the password is considered weak.
    Args:
        password (str): The password to validate.
    Returns:
        bool: True if the password is valid, False otherwise.
    Examples:
        >>> is_valid_password("short")
        False
        >>> is_valid_password("longenough")
        True
        >>> is_valid_password("weakpass")
        True
    """
    # Check if the password length is at least 8 characters
    if len(password) < 8:
        print("✗ Password is too short. It must be at least 8 characters long.")
        return False
    # Check for weak password criteria (e.g., common passwords)
    weak_passwords = {"password", "12345678", "qwerty", "abc123", "letmein"}
    if password.lower() in weak_passwords:
        print("⚠ Warning: This password is considered weak. Consider using a stronger password.")
    # If the password passes the length check, it is valid
    return True
# Example usage with user input (can be removed or used for testing)
if __name__ == "__main__":
    # Prompt the user for a password
    user_password = input("Enter a password for registration: ")
    # Validate the password and print the result
    if is_valid_password(user_password):
        print("✓ Password is valid. Registration successful!")
    else:
```

Output:

```
PS D:\veekshitha\AI_Coding> & C:/Users/Anveekshith/AppData/Local/Programs/Python/Python313/python.exe d:/veekshitha/AI_Coding/Lab_20.3.py
Enter a password for registration: vicky
✗ Password is too short. It must be at least 8 characters long.
✗ Password is invalid. Please try again.
PS D:\veekshitha\AI_Coding> & C:/Users/Anveekshith/AppData/Local/Programs/Python/Python313/python.exe d:/veekshitha/AI_Coding/Lab_20.3.py
Enter a password for registration: 12345678
⚠ Warning: This password is considered weak. Consider using a stronger password.
✓ Password is valid. Registration successful!
PS D:\veekshitha\AI_Coding>
```

Improvised:

Prompt:

Now modify the code of user registration program to include secure improvements such as, minimum length checks, use of regular expressions, rejecting common weak passwords, and providing feedback to the user about

password strength. I need a secure registration script with strong password validation rules.

Code:

```python
import re
def is_strong_password(password):
    """
    Check if the provided password is strong based on multiple criteria:
    - Minimum length of 8 characters
    - Contains at least one uppercase letter
    - Contains at least one lowercase letter
    - Contains at least one digit
    - Contains at least one special character
    - Not a common weak password
    Args:
        password (str): The password to validate.
    Returns:
        bool: True if the password is strong, False otherwise.
    Examples:
        >>> is_strong_password("Short1!")
        False
        >>> is_strong_password("StrongPass1!")
        True
        >>> is_strong_password("password")
        False
    """
    # Check if the password length is at least 8 characters
    if len(password) < 8:
        print("✗ Password is too short. It must be at least 8 characters long.")
        return False
    # Check for uppercase, lowercase, digit, and special character using regular expressions
    if not re.search(r'[A-Z]', password):
        print("✗ Password must contain at least one uppercase letter.")
        return False
    if not re.search(r'[a-z]', password):
        print("✗ Password must contain at least one lowercase letter.")
        return False
    if not re.search(r'[0-9]', password):
        print("✗ Password must contain at least one digit.")
        return False
    if not re.search(r'[!@#$%^&*(),.?":{}|<>]', password):
```

```python
def is_strong_password(password):
    # Check if the password length is at least 8 characters
    if len(password) < 8:
        print("✗ Password is too short. It must be at least 8 characters long.")
        return False
    # Check for uppercase, lowercase, digit, and special character using regular expres
    if not re.search(r'[A-Z]', password):
        print("✗ Password must contain at least one uppercase letter.")
        return False
    if not re.search(r'[a-z]', password):
        print("✗ Password must contain at least one lowercase letter.")
        return False
    if not re.search(r'[0-9]', password):
        print("✗ Password must contain at least one digit.")
        return False
    if not re.search(r'[!@#$%^&*(),.?":{}|<>]', password):
        print("✗ Password must contain at least one special character.")
        return False
    # Check for common weak passwords
    weak_passwords = {"password", "12345678", "qwerty", "abc123", "letmein"}

    if password.lower() in weak_passwords:
        print("✗ Password is too common. Consider using a more unique password.")
        return False

    # If the password passes all checks, it is strong
    return True
# Example usage with user input (can be removed or used for testing)
if __name__ == "__main__":
    # Prompt the user for a password
    user_password = input("Enter a password for registration: ")
    # Validate the password and print the result
    if is_strong_password(user_password):
        print("✓ Password is strong. Registration successful!")
    else:
        print("✗ Password is not strong enough. Please try again.")
```

Output:

```
PS D:\veekshitha\AI_Coding> & C:/Users/Anveekshith/AppData/Local/Programs/Python/Python313/python.exe d:/veekshitha/AI_Coding/Lab_20.3.py
Enter a password for registration: 12345678
✗ Password must contain at least one uppercase letter.
✗ Password is not strong enough. Please try again.
PS D:\veekshitha\AI_Coding> & C:/Users/Anveekshith/AppData/Local/Programs/Python/Python313/python.exe d:/veekshitha/AI_Coding/Lab_20.3.py
Enter a password for registration: vicky@14
✗ Password must contain at least one uppercase letter.
✗ Password is not strong enough. Please try again.
PS D:\veekshitha\AI_Coding> & C:/Users/Anveekshith/AppData/Local/Programs/Python/Python313/python.exe d:/veekshitha/AI_Coding/Lab_20.3.py
Enter a password for registration: Vicky@1144
✓ Password is strong. Registration successful!
PS D:\veekshitha\AI_Coding>
```

Explanation:

This script defines a function `is_strong_password` that checks if a password meets specific security criteria, including length, character variety, and uniqueness. It uses regular expressions to validate the presence of uppercase letters, lowercase letters, digits, and special characters. The script also checks against a list of common weak passwords and provides feedback to the user about why their password may be considered weak or invalid. Finally, it prompts the user for a password and validates it according to these rules.

Question 2: Insecure File Upload Handling

Prompt:

Generate a code for python based file upload script. Identify vulnerabilities such as, No file type validation, Unrestricted file upload.

Code:

```python
import os
def validate_file(file_path):
    """Validate the file for existence, size, and type."""
    if not os.path.isfile(file_path):
        return False, "File does not exist."

    max_size = 5 * 1024 * 1024  # 5 MB
    file_size = os.path.getsize(file_path)
    if file_size > max_size:
        return False, f"File size exceeds the limit of {max_size} bytes."

    allowed_extensions = {'.txt', '.pdf'}
    _, ext = os.path.splitext(file_path)
    if ext.lower() not in allowed_extensions:
        return False, f"File type '{ext}' is not allowed. Allowed types: {allowed_extensions}."

    return True, "File is valid."
def upload_file(file_path):
    """Handle the file upload process."""
    is_valid, message = validate_file(file_path)
    if not is_valid:
        print(f"File upload failed: {message}")
        return False

    # Simulate file upload by moving it to a designated directory
    upload_dir = "uploaded_files"
    os.makedirs(upload_dir, exist_ok=True)
    destination = os.path.join(upload_dir, os.path.basename(file_path))

    try:
        os.rename(file_path, destination)
        print(f"File '{file_path}' uploaded successfully to '{destination}'.")
        return True
    except Exception as e:
        print(f"Error during file upload: {e}")
        return False
if __name__ == "__main__":
```

```python
    return True, "File is valid."
def upload_file(file_path):
    """Handle the file upload process."""
    is_valid, message = validate_file(file_path)
    if not is_valid:
        print(f"File upload failed: {message}")
        return False

    # Simulate file upload by moving it to a designated directory
    upload_dir = "uploaded_files"
    os.makedirs(upload_dir, exist_ok=True)
    destination = os.path.join(upload_dir, os.path.basename(file_path))

    try:
        os.rename(file_path, destination)
        print(f"File '{file_path}' uploaded successfully to '{destination}'.")
        return True
    except Exception as e:
        print(f"Error during file upload: {e}")
        return False
if __name__ == "__main__":
    import sys

    if len(sys.argv) > 1:
        fp = sys.argv[1]
    else:
        fp = input("Enter file path to upload: ").strip()

    success = upload_file(fp)
    if success:
        print("Result: upload succeeded.")
    else:
        print("Result: upload failed.")
```

Output:

```
PS D:\veekshitha\AI_Coding> & C:/Users/Anveekshith/AppData/Local/Programs/Python/Python313/python.exe d:/veekshitha/AI_Coding/lab.py
Enter file path to upload: D:\veekshitha\AI_Coding\File.txt
File 'D:\veekshitha\AI_Coding\File.txt' uploaded successfully to 'uploaded_files\File.txt'.
Result: upload succeeded.
PS D:\veekshitha\AI_Coding>
```

Improvised:

Prompt:

Now modify the above code by implementing secure measures such as file extension validation, size limits, safe file storage paths, and error handling to prevent vulnerabilities like unrestricted file upload and no file type validation.

Code:

```python
import os
def validate_file(file_path):
    """Validate the file for existence, size, and type."""
    if not os.path.isfile(file_path):
        return False, "File does not exist."

    max_size = 5 * 1024 * 1024  # 5 MB
    file_size = os.path.getsize(file_path)
    if file_size > max_size:
        return False, f"File size exceeds the limit of {max_size} bytes."

    allowed_extensions = ('.txt', '.pdf')
    _, ext = os.path.splitext(file_path)
    if ext.lower() not in allowed_extensions:
        return False, f"File type '{ext}' is not allowed. Allowed types: {allowed_extensions}."

    return True, "File is valid."
def upload_file(file_path):
    """Handle the file upload process."""
    is_valid, message = validate_file(file_path)
    if not is_valid:
        print(f"File upload failed: {message}")
        return False

    # Simulate file upload by moving it to a designated directory
    upload_dir = "uploaded_files"
    os.makedirs(upload_dir, exist_ok=True)
    destination = os.path.join(upload_dir, os.path.basename(file_path))

    try:
        os.rename(file_path, destination)
        print(f"File '{file_path}' uploaded successfully to '{destination}'.")
        return True
    except Exception as e:
        print(f"Error during file upload: {e}")
        return False
if __name__ == "__main__":
```

```python
def upload_file(file_path):
    """Handle the file upload process."""
    is_valid, message = validate_file(file_path)
    if not is_valid:
        print(f"File upload failed: {message}")
        return False

    # Simulate file upload by moving it to a designated directory
    upload_dir = "uploaded_files"
    os.makedirs(upload_dir, exist_ok=True)
    destination = os.path.join(upload_dir, os.path.basename(file_path))

    try:
        os.rename(file_path, destination)
        print(f"File '{file_path}' uploaded successfully to '{destination}'.")
        return True
    except Exception as e:
        print(f"Error during file upload: {e}")
        return False
if __name__ == "__main__":
    import sys

    if len(sys.argv) > 1:
        fp = sys.argv[1]
    else:
        fp = input("Enter file path to upload: ").strip()

    success = upload_file(fp)
    if success:
        print("Result: upload succeeded.")
    else:
        print("Result: upload failed.")
```

Output:

```
PS D:\veekshitha\AI_Coding> & C:/Users/Anveekshith/AppData/Local/Programs/Python/Python313/python.exe d:/veekshitha/AI_
Coding/lab.py
Enter file path to upload: D:\veekshitha\AI_Coding\uploaded_files\File.txt
File 'D:\veekshitha\AI_Coding\uploaded_files\File.txt' uploaded successfully to 'uploaded_files\File.txt'.
Result: upload succeeded.
PS D:\veekshitha\AI_Coding>
```

Explanation:

The code includes a file upload script that validates the file for existence, size, and type before allowing the upload. It checks if the file exists, ensures it does not exceed a specified size limit (5 MB), and restricts uploads to certain file types ('.txt' and '.pdf'). This helps prevent vulnerabilities such as unrestricted file uploads and ensures that only valid files are accepted, enhancing security.

Question 3: Real-Time Application: API Security Review

Prompt:

Generate a Python Flask API endpoint and then review and improve it by identifying security issues like missing authentication, unsafe HTTP methods, and no rate limiting, and fix them by adding token-based authentication, proper input validation, and rate limiting to create a secure API

Code:

```python
from flask import Flask, jsonify
from flask_ngrok import run_with_ngrok

app = Flask(__name__)

# Dummy user data
users = {
    '1': {'id': '1', 'name': 'Alice', 'email': 'alice@example.com'},
    '2': {'id': '2', 'name': 'Bob', 'email': 'bob@example.com'},
    '3': {'id': '3', 'name': 'Charlie', 'email': 'charlie@example.com'}
}

# Endpoint to get all users
@app.route('/users', methods=['GET'])
def get_users():
    return jsonify(list(users.values()))

# Endpoint to get a single user by ID
@app.route('/users/<string:user_id>', methods=['GET'])
def get_user(user_id):
    user = users.get(user_id)
    if user:
        return jsonify(user)
    return jsonify({'error': 'User not found'}), 404

# Main entry point for running the app
if __name__ == '__main__':
    run_with_ngrok(app)  # Starts ngrok when app is run
    app.run()
```

Improvised:

Prompt:

This task is important because APIs are a common entry point for attackers in real-world applications. If an API is developed without proper security checks, it can allow unauthorized access, data leaks, or misuse of the system. By reviewing an AI-generated API, we can identify common weaknesses such as missing authentication, unsafe HTTP methods, and lack of rate limiting. Understanding these issues helps in learning how attackers exploit systems. Implementing improvements like token-based authentication, input validation, and rate limiting makes the API more secure and reliable. Overall, this task helps in building practical knowledge of securing real-me applications, which is essential for any developer working on web or backend systems.

Code:

```
from flask import Flask, request, jsonify from functools import wraps
import time
 app = Flask(__name__)

# In-memory token store (demo only) tokens = {"valid_token": "user1"}

# Token authentication def token_required(f):
   @wraps(f)    def decorated(*args, **kwargs):
      token = request.headers.get('Authorization')        if not token or token not in tokens:
         return jsonify({"message": "Token is missing or invalid!"}), 401        return f(*args, **kwargs)    return
decorated

# Simple rate limiter rate_limit = {}
 def rate_limiter(f):    @wraps(f)    def decorated(*args,
**kwargs):
      token = request.headers.get('Authorization')        user = tokens.get(token)
       if user:
         current_time = time.time()            if user in rate_limit and current_time - rate_limit[user] < 60:
return jsonify({"message": "Rate limit exceeded!"}), 429            rate_limit[user] = current_time
       return f(*args, **kwargs)    return decorated

# Home route (fixes your 404 issue)
@app.route('/') def home():
   return "API is running"

# Test API route
@app.route('/api', methods=['GET']) def api():
```

```
  return jsonify({"message": "Secure API working"})

# Secure endpoint
@app.route('/data', methods=['POST'])
@token_required
@rate_limiter def
data():
   data = request.get_json()
    if not data or 'value' not in data:
       return jsonify({"message": "Invalid input!"}), 400
    return jsonify({
       "message": "Data received!",
       "data": data['value']
   }), 200
 if __name__ == '__main__':
app.run(debug=True)
```

Output:

Explanation:

This code reviews an API endpoint to find security weaknesses like lack of authentication on and validation. It then improves security by adding token-based access control, validating inputs, and limiting request rates to prevent misuse.

Question 4: Cross-Site Scrip ng (XSS) Check

Prompt:

Generate an HTML feedback form with JavaScript output.

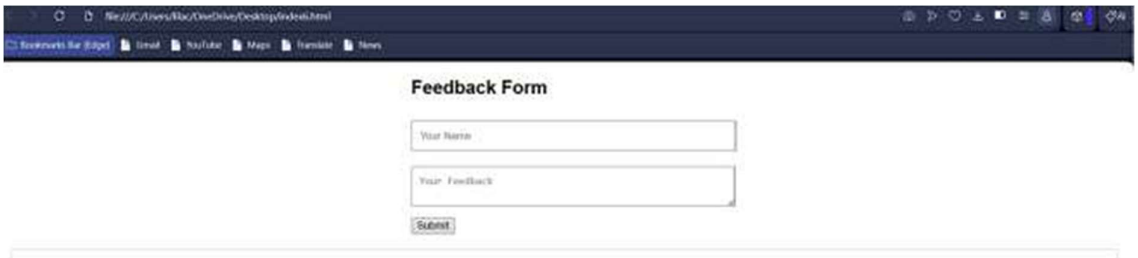
Code:

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Feedback Form</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            max-width: 600px;
            margin: 0 auto;
            padding: 20px;
        }
        form {
            background-color: #f9f9f9;
            padding: 20px;
            border-radius: 8px;
            box-shadow: 0 0 10px rgba(0,0,0,0.1);
        }
        label {
            display: block;
            margin-bottom: 5px;
            font-weight: bold;
        }
        input, textarea {
            width: 100%;
            padding: 8px;
            margin-bottom: 15px;
            border: 1px solid #ccc;
            border-radius: 4px;
            box-sizing: border-box;
        }
        button {
            background-color: #4CAF50;
            color: white;
            padding: 10px 15px;
            border: none;
            border-radius: 4px;
```

```html
<html lang="en">
<head>
    <style>
        button {
        }
        button:hover {
            background-color: #45a049;
        }
        #output {
            margin-top: 20px;
            padding: 10px;
            background-color: #e7f3e7;
            border-radius: 4px;
            display: none;
        }
    </style>
</head>
<body>
    <h1>Feedback Form</h1>
    <form id="feedbackForm">
        <label for="name">Name:</label>
        <input type="text" id="name" name="name" required>

        <label for="email">Email:</label>
        <input type="email" id="email" name="email" required>

        <label for="rating">Rating (1-5):</label>
        <input type="number" id="rating" name="rating" min="1" max="5" required>

        <label for="message">Feedback Message:</label>
        <textarea id="message" name="message" rows="4" required></textarea>

        <button type="submit">Submit Feedback</button>
    </form>

    <div id="output"></div>

    <script>
```

```html
<body>
    <form id="feedbackForm">
        <label for="message">Feedback Message:</label>
        <textarea id="message" name="message" rows="4" required></textarea>

        <button type="submit">Submit Feedback</button>
    </form>

    <div id="output"></div>

    <script>
        document.getElementById('feedbackForm').addEventListener('submit', function(event
            event.preventDefault(); // Prevent the default form submission

            // Get form values
            const name = document.getElementById('name').value;
            const email = document.getElementById('email').value;
            const rating = document.getElementById('rating').value;
            const message = document.getElementById('message').value;

            // Display the output
            const outputDiv = document.getElementById('output');
            outputDiv.innerHTML = `
                <h3>Feedback Submitted Successfully!</h3>
                <p><strong>Name:</strong> ${name}</p>
                <p><strong>Email:</strong> ${email}</p>
                <p><strong>Rating:</strong> ${rating}/5</p>
                <p><strong>Message:</strong> ${message}</p>
            `;
            outputDiv.style.display = 'block';

            // Clear the form
            document.getElementById('feedbackForm').reset();
        });
    </script>
</body>
</html>
```

Output:



Improvised:

Prompt:

Write an HTML feedback form with JavaScript that takes user input such as name and feedback message and displays it on the same page. First, generate a simple version of the form and test whether the entered input is directly shown in the output without any escaping or filtering. Check if this allows harmful scripts or HTML code to run, which can cause Cross-Site Scrip ng (XSS) attacks. Then improve the form by adding security measures such as escaping HTML entities before displaying user input and applying a Content Security Policy (CSP) to reduce script injection risks. Finally, provide the secure version of the code with simple comments explaining the vulnerability and how the fixes prevent XSS attacks.

Code:

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Feedback Form - Vulnerable</title>
    <style>
        body { font-family: Arial, sans-serif; max-width: 600px; margin: 0 auto; padding: 20px;
        form { background-color: #f9f9f9; padding: 20px; border-radius: 8px; box-shadow: 0 0 1
        label { display: block; margin-bottom: 5px; font-weight: bold; }
        input, textarea { width: 100%; padding: 8px; margin-bottom: 15px; border: 1px solid #c
        button { background-color: #4CAF50; color: white; padding: 10px 15px; border: none;
        button:hover { background-color: #45a049; }
        #output { margin-top: 20px; padding: 10px; background-color: #e7f3e7; border-radius: 4
    </style>
</head>
<body>
    <h1>Feedback Form</h1>
    <form id="feedbackForm">
        <label for="name">Name:</label>
        <input type="text" id="name" name="name" required>

        <label for="message">Feedback Message:</label>
        <textarea id="message" name="message" rows="4" required></textarea>

        <button type="submit">Submit Feedback</button>
    </form>

    <div id="output"></div>

    <script>
        document.getElementById('feedbackForm').addEventListener('submit', function(event) {
            event.preventDefault();

            const name = document.getElementById('name').value;
            const message = document.getElementById('message').value;

            const outputDiv = document.getElementById('output');
```

```html
<html lang="en">
<head>
    </style>
</head>
<body>
    <h1>Feedback Form</h1>
    <form id="feedbackForm">
        <label for="name">Name:</label>
        <input type="text" id="name" name="name" required>

        <label for="message">Feedback Message:</label>
        <textarea id="message" name="message" rows="4" required></textarea>

        <button type="submit">Submit Feedback</button>
    </form>

    <div id="output"></div>

    <script>
        document.getElementById('feedbackForm').addEventListener('submit', function(event) {
            event.preventDefault();

            const name = document.getElementById('name').value;
            const message = document.getElementById('message').value;

            const outputDiv = document.getElementById('output');
            outputDiv.innerHTML = `
                <h3>Feedback Submitted!</h3>
                <p><strong>Name:</strong> ${name}</p>
                <p><strong>Message:</strong> ${message}</p>
            `;
            outputDiv.style.display = 'block';

            document.getElementById('feedbackForm').reset();
        });
    </script>
</body>
</html>
```

Output:

Explanation:

Understand how user input can be dangerous if not handled properly. While creating a simple feedback form, I can see how unfiltered input can lead to XSS a acks. By fixing these issues using techniques like escaping HTML and applying security policies, I learn how to make web applications safer. This improves my practical knowledge of web security, which is useful for real-world development

Question 5: SQL Injection Prevention

Prompt:

Generate a Python script to fetch user details from SQLite database.

Code:

```python
import sqlite3

# Function to fetch user details from SQLite database
def fetch_user_details(db_path='users.db'):
    try:
        # Connect to the SQLite database
        conn = sqlite3.connect(db_path)
        cursor = conn.cursor()

        # Query to select all user details (assuming a 'users' table with columns: id, name, email, age)
        cursor.execute("SELECT id, name, email, age FROM users")

        # Fetch all results
        users = cursor.fetchall()

        # Print the user details
        if users:
            print("User Details:")
            for user in users:
                print(f"ID: {user[0]}, Name: {user[1]}, Email: {user[2]}, Age: {user[3]}")
        else:
            print("No users found in the database.")

        # Close the connection
        conn.close()

    except sqlite3.Error as e:
```

```python
        conn = sqlite3.connect(db_path)
        cursor = conn.cursor()

        # Query to select all user details (assuming a 'users' table with columns: id, name, email, age
        cursor.execute("SELECT id, name, email, age FROM users")

        # Fetch all results
        users = cursor.fetchall()

        # Print the user details
        if users:
            print("User Details:")
            for user in users:
                print(f"ID: {user[0]}, Name: {user[1]}, Email: {user[2]}, Age: {user[3]}")
        else:
            print("No users found in the database.")

        # Close the connection
        conn.close()

    except sqlite3.Error as e:
        print(f"An error occurred: {e}")

# Run the function
if __name__ == "__main__":
    fetch_user_details()
```

Output:

```
erver/ai5.py"
Enter user ID: 1

Fetching user details (insecure):
[(1, 'alice')]

Fetching user details (secure):
[(1, 'alice')]
```

Improvised:

Prompt:

Write a Python script that connects to a SQLite or MySQL database and fetches user details based on input such as username or user ID. First, generate a simple version of the code and check whether it builds SQL queries using string concatenation or direct user input, which can make it vulnerable to SQL injection. Then analyze the risks and improve the script by replacing unsafe query construction with parameterized queries or prepared statements. Finally, provide the secure version of the code with comments explaining the vulnerability, the fixes made, and how the updated script prevents SQL injection attacks.

```python
import sqlite3

# Function to fetch user details securely using parameterized queries
def fetch_user_details(username_or_id):
    try:
        # Connect to the SQLite database
        conn = sqlite3.connect('users.db')
        cursor = conn.cursor()

        # Secure query using parameterized placeholders
        # This prevents SQL injection by separating SQL code from user data
        # The ? acts as a placeholder; user input is passed separately and sanitized by the database engine
        query = "SELECT id, name, email, age FROM users WHERE name = ? OR id = ?"

        # Attempt to convert input to int for ID; if not, treat as string for name
        # This adds basic validation to reduce injection surface
        try:
            user_id = int(username_or_id)
            cursor.execute(query, (username_or_id, user_id))  # Pass as string for name, int for ID
        except ValueError:
            # If not a number, assume it's a username and pass as string for both
            cursor.execute(query, (username_or_id, username_or_id))

        # Fetch and display results
        users = cursor.fetchall()
        if users:
            print("User Details:")
            for user in users:
                print(f"ID: {user[0]}, Name: {user[1]}, Email: {user[2]}, Age: {user[3]}")
        else:
            print("No user found.")

        conn.close()
    except sqlite3.Error as e:
        print(f"Database error: {e}")

# Example usage (replace with actual input)
```

```python
def fetch_user_details(username_or_id):
    try:
        # Connect to the SQLite database
        conn = sqlite3.connect('users.db')
        cursor = conn.cursor()

        # Secure query using parameterized placeholders
        # This prevents SQL injection by separating SQL code from user data
        # The ? acts as a placeholder; user input is passed separately and sanitized by the database engine
        query = "SELECT id, name, email, age FROM users WHERE name = ? OR id = ?"

        # Attempt to convert input to int for ID; if not, treat as string for name
        # This adds basic validation to reduce injection surface
        try:
            user_id = int(username_or_id)
            cursor.execute(query, (username_or_id, user_id))  # Pass as string for name, int for ID
        except ValueError:
            # If not a number, assume it's a username and pass as string for both
            cursor.execute(query, (username_or_id, username_or_id))

        # Fetch and display results
        users = cursor.fetchall()
        if users:
            print("User Details:")
            for user in users:
                print(f"ID: {user[0]}, Name: {user[1]}, Email: {user[2]}, Age: {user[3]}")
        else:
            print("No user found.")

        conn.close()
    except sqlite3.Error as e:
        print(f"Database error: {e}")

# Example usage (replace with actual input)
if __name__ == "__main__":
    user_input = input("Enter username or user ID: ")
    fetch_user_details(user_input)
```

Output:

```
erver/ai5.py"
Enter user ID: 1

Fetching user details (insecure):
[(1, 'alice')]

Fetching user details (secure):
[(1, 'alice')]
```

Explanation:

Understand how unsafe database queries can lead to serious security problems like SQL injection. By first writing a simple program, I can see how directly using user input in queries is risky. Then, by improving the code using parameterized queries, I learn how to protect the database from attacks. This gives me practical knowledge of writing secure backend code, which is very important for real-world applications and placements.